

Simple Guide: Steps 00 to 05 with Examples

Step 00: Swarm

What is Swarm AI?

Swarm AI is a fun and experimental tool from Open AI. It helps create smart systems where many small helpers, called agents, work together like a team. Imagine you're building an online customer support system. In this system, each agent has its own job. For example:

- Agent A: Answers simple questions like "How do I log in?"
- Agent B: Fixes billing problems like "My bill is wrong!"
- Agent C: Solves technical issues like "My internet is slow."

So, we have three agents, each with a different task. They all work together, and this teamwork is called Swarm. The main goal of Swarm is to manage these agents easily and make sure everyone knows their job. This keeps things light and simple.

How Does Swarm Work?

Each agent has its own role, and they pass work to each other when needed. This passing of work is called a "handoff."

For example:

- A user asks, "Where is my bill?"
- Agent A (who handles general questions) reads it and thinks, "This is about billing."
- Then, Agent A hands it off to Agent B (the billing expert) to solve it.

This handoff makes sure the right agent handles the right job.

What is Open AI Agent SDK?

The Open AI Agent SDK is like an upgraded version of Swarm. It's more advanced and has extra features. With this tool:

- Agents can automatically figure out tasks and share work.

- They can work at the same time (parallel work), which makes the service faster.

For example, if someone asks for help with a slow internet connection and a new package at the same time, agents can split the work and finish it quickly.

What are Anthropic Design Patterns?

Anthropic is a company that makes safe and helpful AI. They have 5 special ways (design patterns) to build AI systems, and the Open AI Agent SDK uses these to be better than Swarm. Let's look at these 5 patterns with simple examples:

Assume we have a Online Customer Support System

- Agent A: Answers general questions.
- Agent B: Fixes billing issues.
- Agent C: Solves technical and internet problems.
- Agent D: Handles sales and new packages.
- Agent O: The manager who gives tasks. (Orchestrator)
- Agent E: The checker who gives feedback. (Evaluator)

1. Prompt Chaining (Chain Workflow)

This means breaking a big task into small steps and giving each step to a different agent.

Example: A user says, "My internet is slow, and I want a new package."

- Step 1: Agent A sees it's a big task and calls the manager (Agent O).
- Step 2: Agent O splits it:
 - Internet issue goes to Agent C.
 - New package goes to Agent D.
- Result: The task is done step by step like a chain!

2. Routing

This is sending the task to the right agent who knows how to do it.

- Example: A user says, "My bill is wrong."
- Agent A reads it and sends it to Agent B (the billing expert).
- Result: The task goes to the best agent for the job!

3. Parallelization

This means giving work to many agents at the same time to finish faster.

- Example: A user says, “My internet is slow, and my bill is wrong.”
- Agent O or Agent A gives:
 - Billing to Agent B.
 - Internet to Agent C.
- Result: Both tasks are done at once, making it quick!

4. Orchestrator-Workers

This is like having a manager (Orchestrator) who gives tasks to workers (agents).

- Example: A user says, “I need my bill, my internet is slow, and I want a new offer.”
- Agent O (the manager) splits it:
 - Bill to Agent B.
 - Internet to Agent C.
 - Offer to Agent D.
- Result: The manager organizes everything!

5. Evaluator-Optimizer

This is when someone checks the work and gives feedback to make it better.

- Example: After the work is done, Agent E (the checker) looks at it:
 - Did Agent B send the right bill?
 - Did Agent C fix the internet?
 - Did Agent D give the correct offer?
- If someone is slow, Agent E says, “Work faster next time!”
- Result: The system gets smarter and better every time!

Let’s see all these Agents (A, B, C, D, O, E) in action with a simple customer support team:

- Prompt Chaining: A user says, “My internet is slow, and I want a new package.”

- Agent A calls Agent O.
 - Agent O sends internet to Agent C and package to Agent D.
 - Routing: A user says, "My bill is wrong."
 - Agent A sends it to Agent B (billing expert).
 - Parallelization: A user says, "Internet is slow, and bill is wrong."
 - Agent B and Agent C work at the same time.
 - Orchestrator-Workers: A user says, "Bill, internet, and new offer."
 - Agent O gives: Bill to Agent B, internet to Agent C, offer to Agent D.
 - Evaluator-Optimizer: After work, Agent E checks:
 - Is the bill correct? Is the internet fixed? Is the offer right?
 - If slow, Agent E says, "Be faster next time!"
-

Step 01: UV

What is UV?

UV is a super-fast tool for managing Python projects from start to finish. Think of it like a "superhero" version of older tools like pip or Poetry. It helps you create projects, install packages (like libraries you need for your code), manage different versions of Python, and keep everything organized and reproducible (meaning it works the same way on your computer as on someone else's).

Why is UV great for beginners?

- It's 10-100 times faster than traditional tools like pip.
- It handles everything in one place: creating environments, adding dependencies, locking versions for safety, running code, and even building apps.
- No more headaches with slow installs or mismatched versions.
- It's written in Rust (a fast programming language), so it's efficient and reliable.

UV works with two main files:

- **pyproject.toml**: This is where you list what your project needs (like dependencies and settings).

- **uv.lock:** This locks exact versions of packages so everyone gets the same setup.

UV supports two types of projects:

- **Applications:** Programs you run, like CLI tools or scripts.
- **Libraries:** Reusable code that other projects can import.

For beginners, start with applications—they're easier to understand and run.

Applications vs. Libraries in UV

- **Applications:** These are ready-to-run programs, like a game, a web app, or a simple script. They have a starting point (like a "main" function) and run in their own environment. Example: A tool that fetches weather data and prints it.
- **Libraries:** These are building blocks for other code. You don't run them directly; you import them into apps. Example: A library with math functions that many apps can use.

Both are useful—apps give you something to run and see results, while libraries help you share code. As a beginner, focus on apps first to see quick results.

How Applications Are Made in UV

UV lets you create apps in two ways:

- **Packaged Applications:** These are structured like real software packages. Good for sharing, versioning, or long-term projects.
- **Simple Applications:** Quick folders with scripts—no fancy packaging. Perfect for testing ideas or small tools.

Both ways are easy, but packaged ones are better as your project grows.

Development Lifecycle (Step-By-Step)

Here's how a typical project flows with UV. I'll explain each step simply:

1. **Initialize the Project:** Use `uv init` to create a basic folder structure. This sets up files like `pyproject.toml` automatically—no manual work!
2. **Pin the Python Version:** Use `uv python pin 3.12` to lock a specific Python version. UV downloads it if needed, so everyone uses the same one.

3. **Create Environment:** UV makes a virtual environment (a safe space for your project) automatically when you add packages. No sharing global Python messes.
4. **Add Runtime Dependencies:** Use `uv add package-name` to add libraries your app needs to run. UV updates files and installs fast.
5. **Add Dev Tools:** Use `uv add --dev tool-name` for extras like testers or formatters (e.g., Ruff for code checks).
6. **Optional Feature Groups:** Use `uv add --group group-name package` for add-ons, like docs tools.
7. **Lock Dependencies:** UV creates `uv.lock` to freeze versions—great for teamwork.
8. **Sync Environment:** `uv sync` makes sure your setup matches the lock file.
9. **Run Code & Tools:** `uv run command` runs everything in the right environment—no activation needed.
10. **Lint / Format / Type Check:** Run tools like `uv run ruff check` to keep code clean.
11. **Test:** `uv run pytest` runs tests with locked dependencies.
12. **Upgrade Dependencies:** `uv lock --upgrade` updates packages safely.
13. **Build Artifacts:** `uv build` creates files to share your app.
14. **Publish:** `uv publish` uploads to places like PyPI.
15. **Use Cache:** UV caches downloads for speed—clean with `uv cache prune`.
16. **Offline / Restricted:** Work without internet using cached files.
17. **CI Reproducibility:** Use `uv sync --frozen` in tools like GitHub Actions.
18. **Inspect Environment:** `uv run pip list` shows installed packages.
19. **Clean & Prune:** Remove old stuff to save space.
20. **Monorepo Management:** Handle multiple projects in one folder.
21. **Migrate (Optional):** Import old `requirements.txt` files with UV.

Common Beginner Pitfalls (and Fixes)

- **Forgetting to commit `uv.lock`:** Always add it to Git—it's like a recipe for exact setups. Fix: Run `git add uv.lock` before committing.
- **Editing `pyproject.toml` by hand:** Use `uv add` instead—it auto-updates the lock. Fix: After edits, run `uv sync`.
- **Manually activating environments:** UV's `uv run` handles it. Fix: Skip activation unless needed for editors.

- **Mixing pip and UV:** Stick to UV commands. Fix: If you use pip accidentally, run `uv sync` to fix.
- **Slow starts:** UV is fast, but clear cache if issues: `uv cache clean`.

Python Installation (Step-by-Step for All OS)

Before UV, install Python. It's free and easy!

Windows:

1. Go to <https://www.python.org/downloads/windows/> and download the latest (3.x.x).
2. Run the .exe file. Check "Add Python to PATH" box.
3. Install and verify: Open Command Prompt, type `python --version`.

macOS:

1. Install Homebrew: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`.
2. Then: `brew install python`.
3. Verify: `python3 --version`.

Linux (Ubuntu):

1. `sudo apt update`.
2. `sudo apt install python3 python3-pip -y`.
3. Verify: `python3 --version`.

How to Install UV?

Two easy ways:

Method 1 (Recommended - Standalone Installer):

- Windows: Open PowerShell, run `powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"`.
- macOS/Linux: `curl -Lsf https://astral.sh/uv/install.sh | sh`.
- Verify: `uv --version`.

Method 2 (Via pip):

- Run `pip install uv`.

- Verify: `uv --version`.

Docs: <https://docs.astral.sh/uv/>

METHOD 1

Step-1 : UV DOCS : <https://docs.astral.sh/uv/>

Step-2 : Go to installation tab.

Step-3 : Copy powershell uv installation command

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Step-4 : Open Powershell (windows powershell in your system) and run the copied command

Step-5 : Initialize project : `uv init` .

Step-6 : Add Package : `uv add <package-name>` (for example `uv add openai-agents`)

Step-7 : `uv venv .venv\Scripts\activate`

OR

METHOD 2

Ctrl + Window + R

type Cmd

write pip install uv

uv is successfully installed in your system

then check uv version

`uv --version`

Creating Projects with UV (Two Methods)

Method 1: Manual Folder Creation

1. Create a folder manually (e.g., right-click > New Folder).
2. Open it in VS Code/Cursor.
3. In terminal: `uv init .` (dot for current folder).

```
C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>uv init .
Initialized project `age3ntic-ai` at `C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI`
```

4. Create virtual env: `uv venv`.

```
C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>uv venv
Using CPython 3.13.2 interpreter at: C:\Users\System Angel\AppData\Local\Programs\Python\Python313\python.exe
Creating virtual environment at: .venv
Activate with: .venv\Scripts\activate
```

```
C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>
```

5. Activate: `.venv\Scripts\activate` (Windows) or `source .venv/bin/activate` (macOS/Linux).

```
C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>.venv\Scripts\activate
(Age3ntic AI) C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>
```

6. Add package: `uv add openai-agents`.


```
(Age3ntic AI) C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>uv add openai-agents
Resolved 39 packages in 8.79s
.: Preparing packages... (3/4)
openai ----- 587.00 KiB/769.29 KiB
```

7. Run: `uv run main.py`.

```
(Age3ntic AI) C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>uv run main.py
Hello from age3ntic-ai!

(Age3ntic AI) C:\Users\System Angel\OneDrive\Desktop\Age3ntic AI>
```

Method 2: Direct from Terminal

1. In terminal: `mkdir my-project && cd my-project`.
2. `uv init .`
3. Follow steps 4-7 from Method 1.

Simple Application with UV

Best for quick scripts.

Prerequisites: UV and Python installed.

Steps:

1. `uv init my-simple-app && cd my-simple-app`.
2. `uv sync` (creates env and lock).
3. Open in VS Code: `code .`
4. Select interpreter: `Ctrl+Shift+P > Python: Select Interpreter > Pick .venv one`.
5. Edit `main.py`: Add `print("Hello!")`.
6. Run: `uv run python main.py`.
7. Optional script in `pyproject.toml`: Add `[project.scripts] myapp = "main:main"` and `[tool.uv] package = true`. Then `uv sync` and `uv run myapp`.

Tips: Add deps with `uv add`. Commit key files.

Packaged Application with UV

For structured, shareable apps.

Steps:

1. `uv init --package my-packaged-app && cd my-packaged-app.`
2. `uv sync.`
3. Open in VS Code, select interpreter. Edit `src/my_packaged_app/init.py` or add `main.py`.
4. Add script in `pyproject.toml`: `[project.scripts] myapp = "my_packaged_app:main".`
5. Run: `uv run myapp` or `uv run -m my_packaged_app.main.`
6. Add deps: `uv add package.`

Tips: Keep `init.py` simple. Use for CLI apps.

Quick Commands

- `uv init .` : Initialize project.
- `uv venv` : Create virtual environment.
- `.venv\Scripts\activate` : Activate virtual environment
- `uv add package-name`: Add dependencies.
- `uv run main.py` command: Run file.

Documentations

- [How to install UV - Notion Guide?](#)
- [Python UV: The Ultimate Guide to the Fastest Python Package Manager](#)
- [Official Docs](#)
- [Running scripts](#)
- [Working on projects](#)
- [CLI Reference](#)

Watch: <https://www.youtube.com/watch?v=-J5SnWR4UXw> for a demo.

Step 02: What is API

APIs

If you're new to programming or tech, don't worry—APIs might sound fancy, but they're actually super simple once you break them down. We'll go step by step, with examples, analogies (like real-life stories), and even code you can try yourself. By the end, you'll feel confident about what APIs are and how to use them.

Think of this as a beginner's roadmap. We'll cover:

- What an API really is.
- Fun analogies to make it stick.
- Real examples with code (from weather to cat facts to AI chats).
- How to try them yourself.
- Comparisons between different API types (like OpenAI's options).
- Tips for starting out.

So,

1. What is an API?

Big Idea: API stands for "Application Programming Interface." It's just a set of rules that lets one piece of software talk to another. In simple terms, it's like a bridge between apps or services. You ask for something (like data), and the API gives it back in a neat, organized way.

Why do we need APIs? Imagine you want weather info for your city. Instead of building a whole weather system yourself (which would take forever), you use someone else's ready-made service through their API. It's fast, reliable, and saves you tons of work.

Explanation: An API is like a "menu in a restaurant."

- You (the customer) don't go into the kitchen to cook.
- You look at the menu (the API rules), pick what you want (like "burger"), and tell the waiter (the API request).
- The kitchen (the server) makes it and sends it back through the waiter (the API response).

- Bonus: If the kitchen changes recipes or tools, you don't care—as long as the menu stays the same, you get your burger!

This keeps things simple and separate. APIs are everywhere: in apps, websites, and even smart devices.

More Everyday Analogies (to make it fun and memorable):

Here's a quick table with real-life examples:

Situation	"Kitchen" (The Server/Backend)	"Waiter/API" (The Bridge)	"You/Client" (The User/App)
Weather App	Weather company's big computers storing data	Weather API (rules for asking about temperature, rain, etc.)	Your phone app asking "What's the weather in London?"
Ride-Hailing App (like Uber)	Company's servers tracking drivers and maps	Trip-Booking API (rules for requesting a ride)	Rider's phone app saying "Pick me up here!"
Online Store Checkout	Payment company's secure servers (like Stripe or PayPal)	Payment API (rules for charging a card)	Store website saying "Charge \$10 for this item."

These analogies show APIs hide the complicated stuff (like how the kitchen works) and just give you what you need.

Extra Tip: APIs are "interfaces" because they define how to interact, not what happens behind the scenes. They're public (anyone can use them with rules) or private (only for specific users).

2. What Does an API Look Like in Practice?

APIs aren't scary—they're just web addresses (URLs) you "call" to get or send data. Most use simple methods like:

- GET: Ask for data (like reading a menu).
- POST: Send data (like placing an order).

Data comes back as JSON (like a simple list: `{"key": "value"}`)—easy to read). We'll use Python code here, but you can try these in your browser or a tool like

Postman (free app for testing APIs). Postman is great for beginners: Download it, make a new request, paste the URL, and hit "Send." It even gives you code snippets to copy!

Here are three live examples. I'll explain each, show the code, and what to expect.

Example 1: Get Current Weather for London (Simple GET Call)

- What it does: Asks a free weather service for today's info in London.
- Why easy? No login needed—just a URL.
- Try in Browser: Paste the URL and hit Enter.

Code (Python):

```
import requests # This library helps make API calls
import pprint   # Makes the output pretty and readable

url = "https://api.open-meteo.com/v1/forecast?latitude=51.5&longitude=-0.12&current_weather=true"
resp = requests.get(url, timeout=10) # Send the request (GET method)
resp.raise_for_status()               # Check if it worked (no errors)
weather = resp.json()                 # Get the response as JSON
pprint.pp(weather)                   # Print it nicely
```

Expected Output (something like this—changes with real weather):

```
{'latitude': 51.5,
 'longitude': -0.120000124,
 'generationtime_ms': 0.06592273712158203,
 'utc_offset_seconds': 0,
 'timezone': 'GMT',
 'timezone_abbreviation': 'GMT',
 'elevation': 1.0,
 'current_weather_units': {'time': 'iso8601',
                           'interval': 'seconds',
                           'temperature': '°C',
                           'windspeed': 'km/h',
                           'winddirection': '°',
                           'is_day': '',
                           'weathercode': 'wmo code'},
 'current_weather': {'time': '2025-07-20T14:15',
                     'interval': 900,
                     'temperature': 21.2,
                     'windspeed': 15.2,
                     'winddirection': 202,
                     'is_day': 1,
                     'weathercode': 3}}
```

What it means: Temperature is 21.2°C, wind speed 15.2 km/h, etc. Super simple!

In Postman: New > GET > Paste URL > Send.

Extra Tip: Change "latitude" and "longitude" in the URL for your city (Google your coords). This shows how APIs use "parameters" (like ?latitude=...) to customize requests.

Example 2: Fetch a Random Cat Fact (Another GET Call)

- What it does: Gets a fun fact about cats from a free API.
- Why fun? Random each time—great for testing.

Code (Python):

```
import requests
import pprint

url = "https://catfact.ninja/fact"
resp = requests.get(url, timeout=10)
resp.raise_for_status()
cat_fact = resp.json()
pprint.pp(cat_fact)
```

Expected Output (example—it's random):

```
{'fact': 'Mohammed loved cats and reportedly his favorite cat, Muezza, was a
tabby. Legend says that tabby cats have an "M" for Mohammed on top of their
heads because Mohammad would often rest his hand on the cat's head.',
'length': 176}
```

In Postman: Same as above—GET, paste URL, Send.

Extra Tip: APIs often limit how many calls you can make (rate limits). This one's free and unlimited for fun, but real ones might need a key (like a password).

Example 3: OpenAI Chat Completions (POST Call with AI)

- What it does: Asks an AI (like ChatGPT) a question and gets a smart answer.
- Why cool? Shows "conversations" with machines.

- Note: Needs an OpenAI account and API key (free trial available). Set it up at openai.com/api.

This API is "stateless" (no memory—you send the whole chat history each time).

Code (Python):

```
from openai import OpenAI # Install with: pip install openai

client = OpenAI() # Add your API key: OpenAI(api_key="your-key-here")

response = client.chat.completions.create(
    model="gpt-4o-mini", # Cheap, fast model
    messages=[{"role": "user", "content": "What's the capital of France?"}]
)
print(response.choices[0].message.content) # Output: Paris
```

Expected Output: "Paris" (or a full sentence).

To make it remember (stateful-like): Add past messages.

Extra Tip: "Role" is like who's talking: "user" (you), "assistant" (AI), "system" (instructions like "Be helpful").

3. OpenAI's Responses API (Stateful, with Tools)

This is like Chat Completions but "smarter"—it remembers chats automatically and has built-in tools (like web search).

Code Example:

```
from openai import OpenAI

client = OpenAI()

# First call: Ask with a tool (web search)
r1 = client.responses.create(
    model="gpt-4o-mini",
    input="Search the latest AI news in healthcare.",
    store=True, # Remember this chat
    tools=[{"type": "web_search_preview"}] # Turn on search tool
)

# Follow-up: No need to resend history!
r2 = client.responses.create(
    previous_response_id=r1.id, # Link to previous
    input="Summarize those findings in bullet points."
)

print(r2.response.text) # Output: Bullet points from search results
```

Why Stateful? Server keeps history—you just send new input.

Built-in Tools: Search web, generate images, run code, etc.—turn on with one line.

Extra Tip: Great for bots that "remember" without you managing it.

4. Quick Side-by-Side Comparison

Feature	Chat Completions (Stateless)	Responses (Stateful)
Memory	You send history each time	Server remembers
Built-in Tools	Manual (write your own)	Easy one-line opt-in
Typical Use	Simple chats, full control	Smart assistants
Follow-ups	Resend all messages	Just previous ID
Code Boilerplate	More	Less

5. “Which One Should I Start With?”

- Learn Basics: Chat Completions (see the full "message flow").
- Build a Smart Bot: Responses (easy memory + tools).
- Bottom Line: Start with Chat Completions for concepts. Switch to Responses for real apps.

6. Bonus - Next Steps for the Curious

- Try the weather API in Postman: New GET > Paste URL > Send > Click "Code" > Copy Python snippet > Run in a file.
- Swap URLs: Try cat facts, then a joke API.
- OpenAI: Sign up, get a free key, try Chat Completions in Postman (POST to <https://api.openai.com/v1/chat/completions>, add headers for key).
- Common Mistake Fix: If "timeout" or errors—check internet or URL typos.

Extra: Practice with free APIs (search "public APIs list"). Start small—build a weather bot! If stuck, Google "API error [your issue]" or ask on forums like Stack Overflow.

APIs make coding fun and powerful. You've got this—try one example today! If questions, feel free to ask.

Step 03: Get API Key

In this step, we'll talk about API keys. These are like special passwords that let your code connect to services like AI tools (e.g., OpenAI or Google Gemini). Some API keys are free (with limits), and some might need payment for more use. Don't worry—getting them is easy, and I'll explain everything in simple steps. We'll cover three popular ones: OpenAI, Google Gemini, and OpenRouter. I'll add some extra tips to make it clearer, like why security matters and common mistakes to avoid.

At the end, I'll explain how to use your API key in a Python project safely.

Why Do You Need an API Key?

- It proves you're allowed to use the service.
- It tracks your usage (e.g., how many questions you ask the AI).
- Free keys often have limits (like 10 requests per day), but paid ones give more.
- Extra Tip: Start with free tiers to test—most give you credits to try without paying right away.

Security Reminder (Super Important!)

- Never share your key online (e.g., don't post it on GitHub or forums).
- If someone steals it, they could use your account and run up costs.
- Use safe ways to store it (we'll cover this in "How to Use It").
- If you think it's leaked, delete and make a new one.

Quick Reference Table

Here's a simple table to compare the providers:

Provider	Website	API Key Link
OpenAI	https://platform.openai.com/	https://platform.openai.com/api-keys
Google Gemini	https://aistudio.google.com/app/apikey	https://aistudio.google.com/app/apikey
OpenRouter	https://openrouter.ai/	https://openrouter.ai/settings/keys

Now, let's get the keys step by step.

1. Getting an OpenAI API Key

OpenAI lets you use powerful AI models like GPT-4 (for chatbots or writing help). It's free to start, but you might add payment for more use.

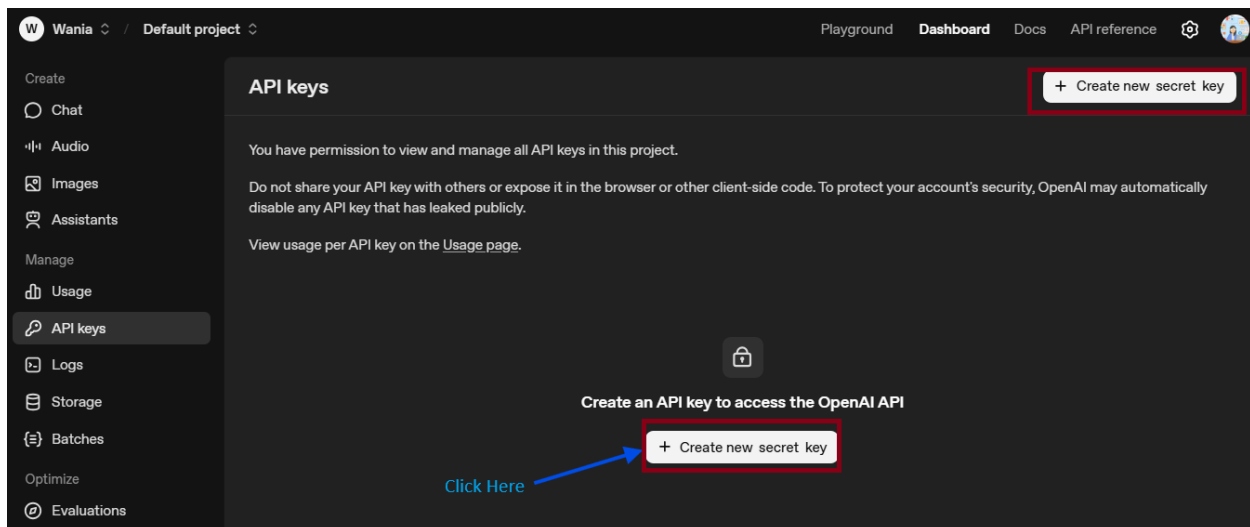
Steps:

1. Sign Up or Log In:

- Go to <https://platform.openai.com/>.
- Click "Sign Up" if new (use email or Google), or "Log In" if you have an account.

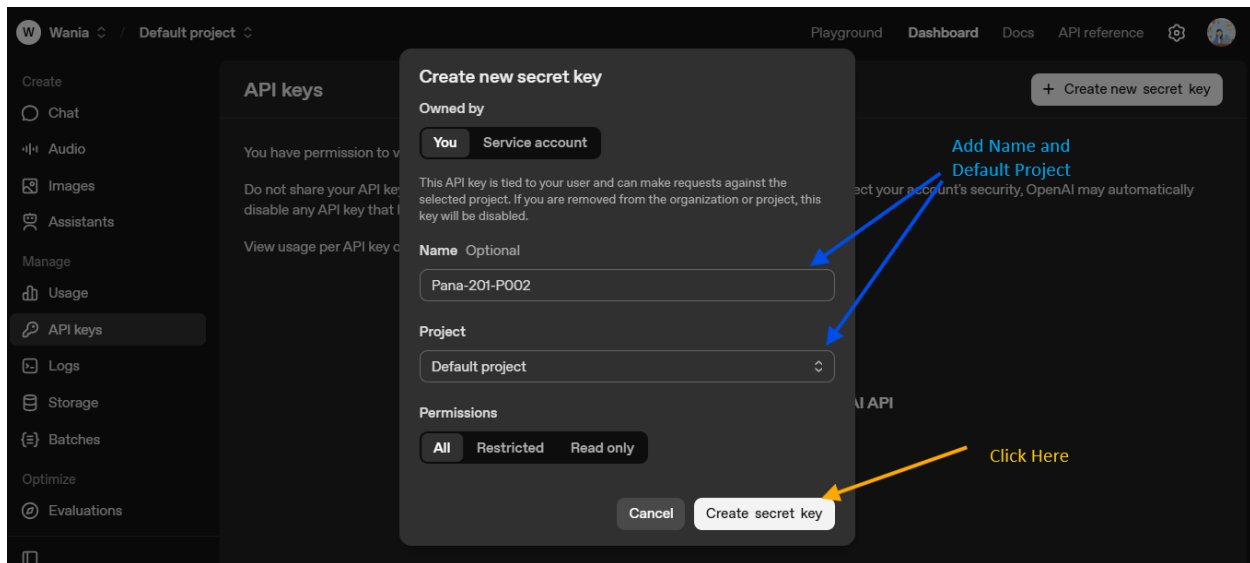
2. Navigate to API Keys:

- Once logged in, click the settings icon (gear wheel).
- Choose "API keys" from the menu, or go straight to <https://platform.openai.com/api-keys>.

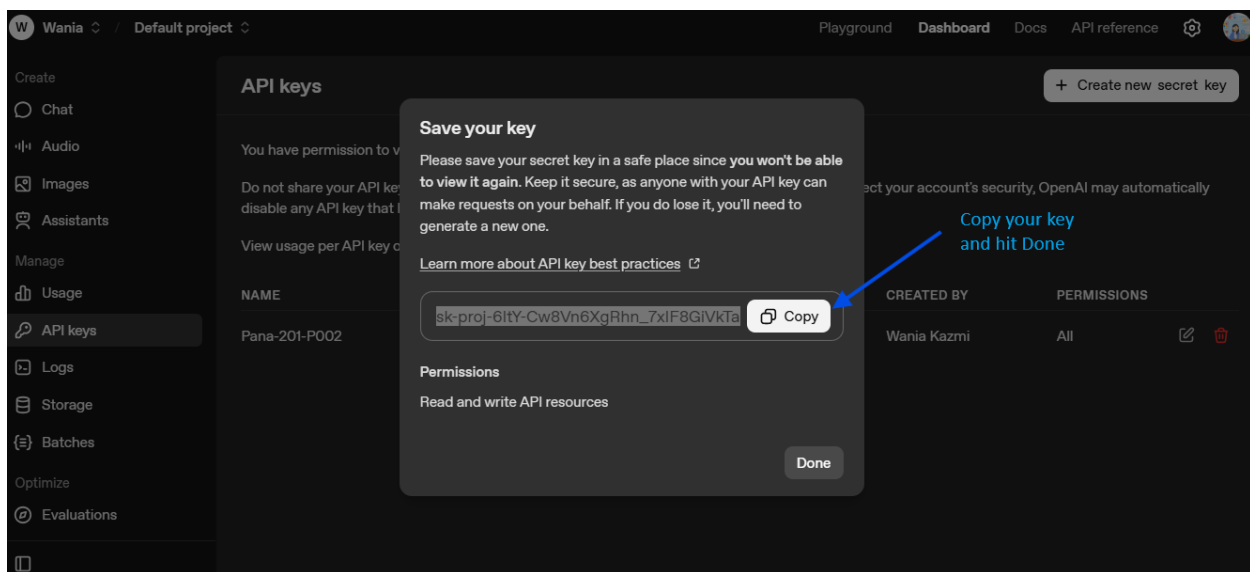


3. Create a New API Key:

- Click the "+ Create new secret key" button.
- Give it a name (like "My Project Key") to remember what it's for.



- Click "Create" and copy the key that appears (it starts with "sk-").



- Warning: You can't see this key again—save it right away in a safe note!

Notes:

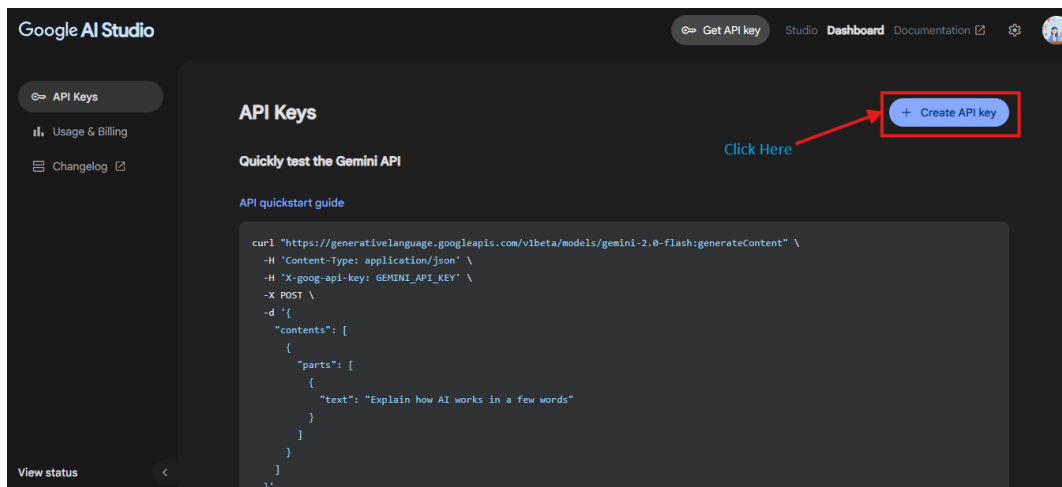
- If it asks for payment, add a card for limits (starts with \$5-10 free credits).
- Check OpenAI docs for help: <https://platform.openai.com/docs/>.
- Extra Tip from Me: Test your key with a simple code (we'll see how later). If it doesn't work, make sure your account has credits.

2. Getting a Google Gemini API Key

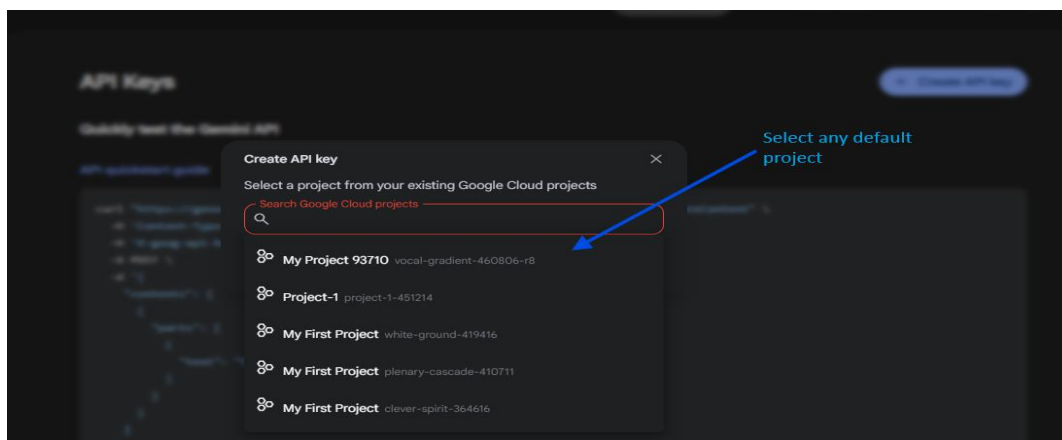
Gemini is Google's AI (like ChatGPT but from Google). It's free for basic use and great for quick tests.

Steps:

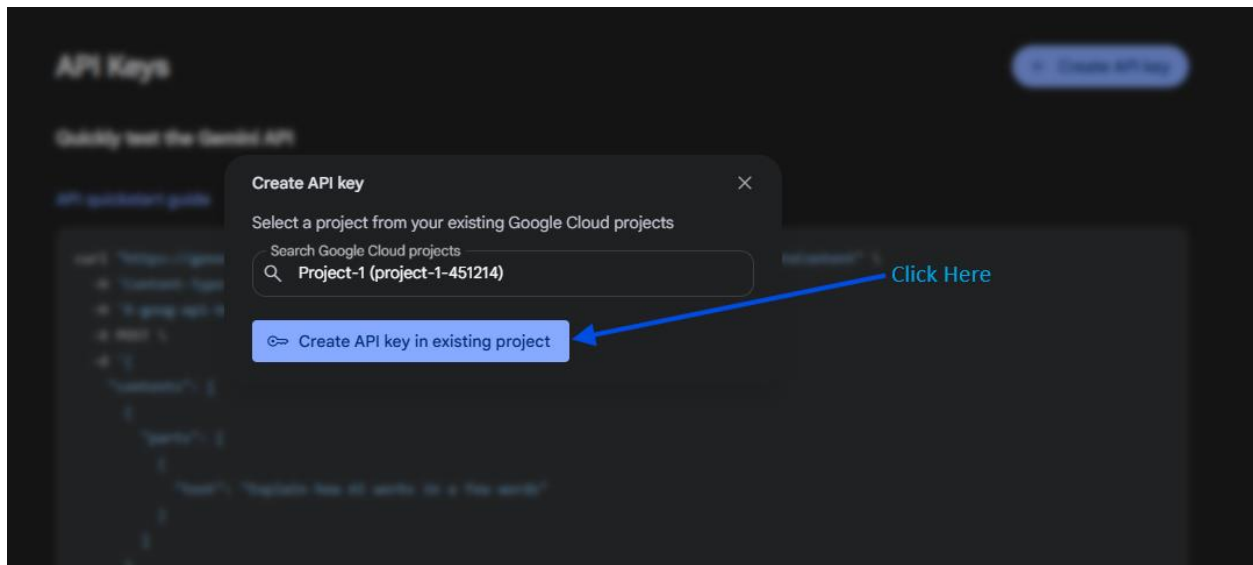
1. **Go to Google AI Studio:**
 - Visit <https://aistudio.google.com/app/apikey>.
2. **Sign In with Google:**
 - Click "Sign In" and use your Google account (Gmail).
 - If no account, make one—it's free.
3. **Create a Gemini API Key:**
 - On the page, click "Create API key".



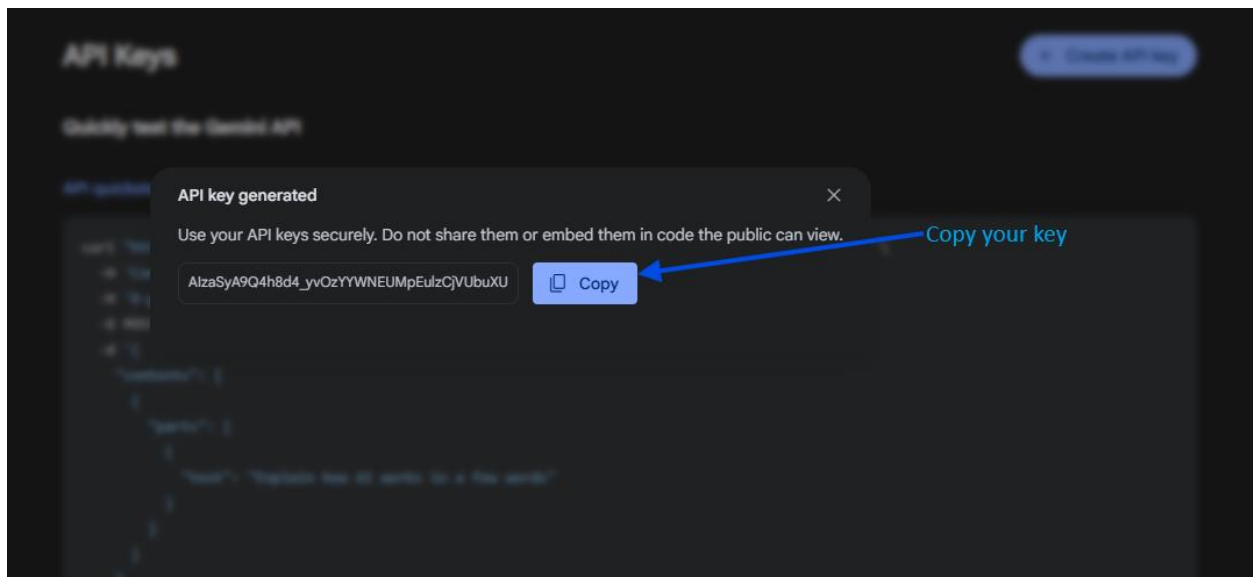
- Pick a default project (or create one if asked—it's like a folder for your keys).



- Click "Generate" or "Create".



- Copy the key (it starts with "Alza").



4. Enable Billing (If Needed):

- For more use, it might ask to turn on billing (add a card in Google Cloud Console).
- Start free—no charge until you hit limits.

Notes:

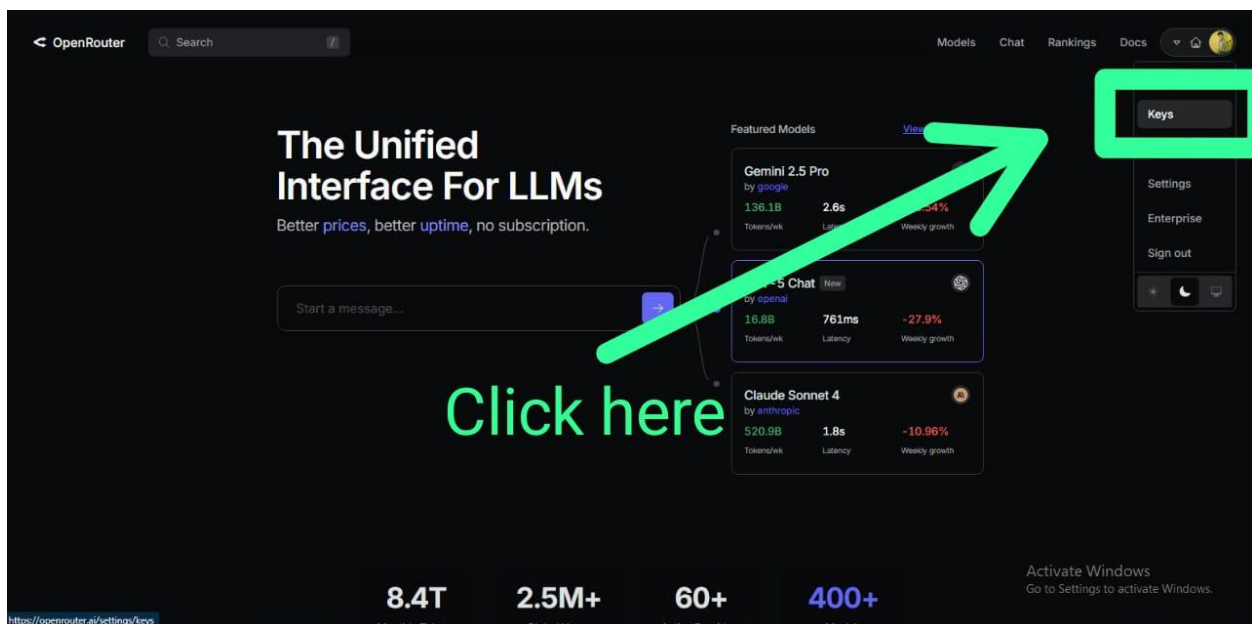
- Keep it private!
- More info in docs: <https://ai.google.dev/gemini-api/docs/quickstart>.
- Extra Tip: Gemini is good for beginners because it's tied to your Google account—no extra setup.

3. Getting an OpenRouter API Key

OpenRouter lets you use many AI models (from different companies) in one place. It's like a "menu" for AIs.

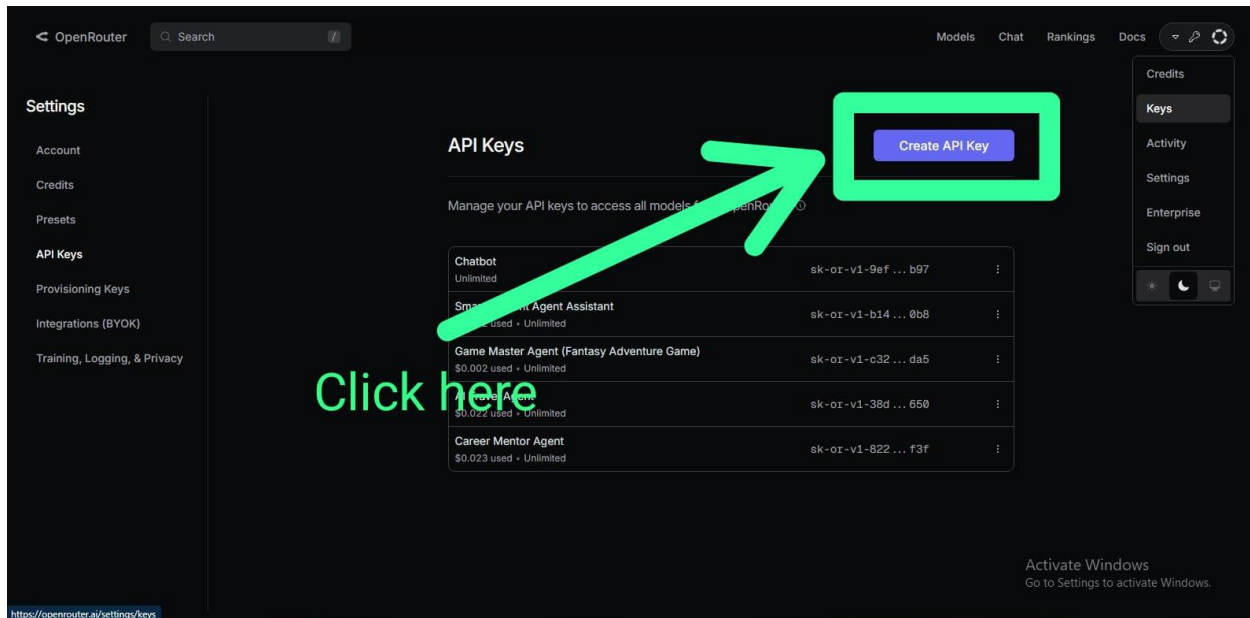
Steps:

1. **Go to OpenRouter:**
 - Visit <https://openrouter.ai/>.
2. **Sign In:**
 - Click "Sign In" and use Google or GitHub.
 - If new, it creates an account automatically.
3. **Navigate to API Keys:**
 - Click the toggle or menu in the top right.
 - Select "Keys" or go to <https://openrouter.ai/settings/keys>.

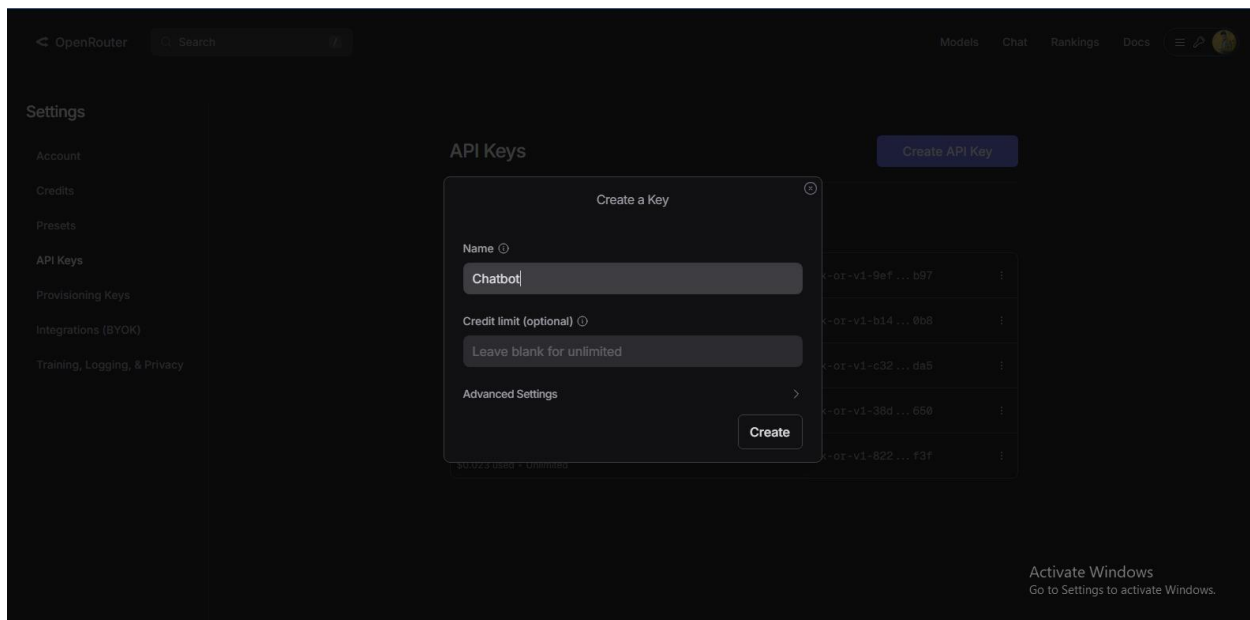


4. Create a New API Key:

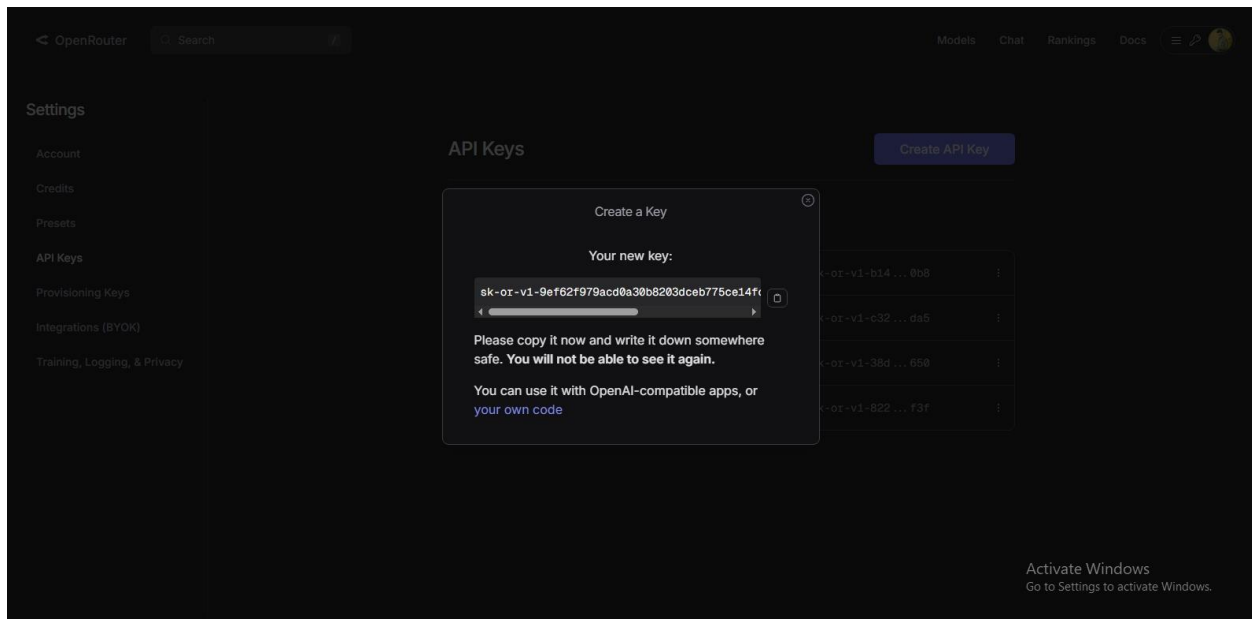
- Click "Create API Key".



- In the first box, give it a name (like "Test Key").
- Leave the second box blank for unlimited credits (or set a limit if you want).



- Click "Create" and copy the key.



Notes:

- Free to start, but add payment for heavy use.
- Docs: <https://openrouter.ai/docs/quickstart>.
- Extra Tip from Me: OpenRouter is flexible—try different models without new keys. Great if you want options.

If issues: Check docs or retry. Common mistake: Wrong browser—use Chrome if stuck.

How to Use Your API Key in a Project

Now that you have the key, let's use it safely in Python code. We'll use UV (from earlier steps) to set up, and a file called ".env" to hide the key. This way, it's not in your code (safer!).

Why This Way? Direct keys in code can leak (e.g., if you share on GitHub). .env keeps it secret.

Steps (Simple and Step-by-Step):

1. Initialize Your Project with UV:

- Open your terminal (in VS Code: Ctrl+` or Terminal > New Terminal).
- Go to your project folder (or make one: `mkdir my-project && cd my-project`).
- Run: `uv init .` (dot means current folder). This sets up your project.
- (Extra Tip: If UV not installed, see Step 01.)

2. Create a .env File:

- In your project folder, make a new file called ".env" (use VS Code: Right-click > New File).
- Inside, write your key like this (pick the right name):
 - For Gemini: `GEMINI_API_KEY="your-key-here"`
 - For OpenAI: `OPENAI_API_KEY="your-key-here"`
 - For OpenRouter: `OPENROUTER_API_KEY="your-key-here"`
- Save it. (Don't commit .env to Git—add to .gitignore.)
- (Picture Idea: Screenshot of .env file open in editor.)

3. Add the Dotenv Package:

- In terminal: `uv add python-dotenv`.
- This adds a tool to load your .env secrets.
- (Extra Tip: UV installs it fast and updates your project files.)

4. Use the Key in Your Code:

- In your main Python file (e.g., main.py), add these at the top:

```
from dotenv import load_dotenv  # Loads .env file
import os                      # Built-in Python tool for secrets

load_dotenv()  # Read the .env file

# Now get your key (example for OpenAI)
my_key = os.getenv("OPENAI_API_KEY")  # Safe way to grab it
print(my_key)  # Test: Prints your key (don't do in real code!)
```

- Run: `uv run main.py` (UV handles the environment).
- Why os? It's Python's way to talk to your computer's settings safely. It comes pre-installed with Python.

Full Example Code (Testing OpenAI):

```
from dotenv import load_dotenv
import os
from openai import OpenAI # Add with: uv add openai

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

client = OpenAI(api_key=api_key)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello!"}]
)
print(response.choices[0].message.content) # AI says hi back!
```

Extra Tips:

- Test Small: Run code with a simple question to check the key works.
- Common Mistake: Typos in .env (e.g., wrong name)—double-check!
- For Teams: Share code without keys—everyone makes their own .env.
- Advanced: Use secrets managers (like AWS) for big projects.

You've got your key and know how to use it safely! Practice by running the example. If stuck, check the repo or docs.

Step 04: Hello Agent (Create Your First AI Agent)

In this step, we're going to create your very first AI agent using the OpenAI Agents SDK. Don't worry if it sounds advanced—I'll explain everything in super simple words. We'll go step by step, with easy examples, code you can copy, and fun analogies. I've added some extra tips to make it even clearer (like common mistakes and why things work). By the end, you'll have a working agent that can do something cool, like translating sentences.

Practice is key—copy the code into VS Code and run it!

What is OpenAI Agents SDK?

Imagine you want to build a smart helper—like a chatbot or AI robot—that can:

- Answer questions cleverly.
- Use tools (like a calculator or web search).
- Ask another AI for help if needed.

The OpenAI Agents SDK is a free Python library that makes this easy. It's like a "toolkit" for creating AI "agents" (smart programs powered by models like GPT or Gemini).

In Simple Words:

- **Agent:** A smart AI "person" with a job (instructions you give it).
- **Tool:** Something the agent can use, like a file reader or math solver.
- **Handoff:** When one agent passes the task to another (like "Hey, expert, take over!").
- **Guardrail:** A safety check to block bad inputs (like naughty questions).
- **Runner:** The "engine" that runs the agent's brain and gets results.

Analogy for a 5-Year-Old (Fun Way to Remember):

- Think of a big school with teachers.
 - **Math Teacher** (Agent): Helps with numbers.
 - **History Teacher** (Another Agent): Helps with old stories.
 - **Receptionist** (Main Agent): Decides who to send the student to.
- The student (you) asks a question. The receptionist hands it off to the right teacher. The teacher might use a "tool" like a book. If the question is silly, a guardrail stops it.
- Everything is logged (tracing) so you can check what happened.

Hello World Example (Quick Test Code):

Here's a simple code to make an agent write a poem. (Copy this into a file like main.py.)

```

from agents import Agent, Runner # Import the tools

agent = Agent(
    name="Poet", # Name of the agent
    instructions="You are a fun poet. Write short poems." # What it does
)

result = Runner.run_sync(agent, "Write a haiku about recursion in programming.") # Run it
print(result.final_output) # Show the answer

```

Output (Example):

```

Code within the code,
Functions calling themselves now,
Infinite loop's dance.

```

Why Use It?

- Super easy to learn and code.
- Builds real AI apps (like chatbots or helpers).
- Has built-in features for tools, handoffs, safety, and debugging (tracing shows every step).
- Free and works with models like GPT or Gemini.
- Extra Tip from Me: Start small—it's like Lego blocks for AI. As you practice, add tools (e.g., make it search the web).

Create Your First AI Agent

STEPS:

1. Install UV
2. Create Folder
3. Create Virtual Environment
4. Install Python Env Package
5. Create API Key
6. Apply API Key in .env
7. Install Open AI Agents
8. Create Your First Agent
9. Connection with Your LLM
10. Execute/Run the Agent

I mentioned 10 steps. You've already done the first 6 (like installing UV, creating folder/env, getting API key, adding to .env). Now, focus on the last 4: Install OpenAI Agents, Create Agent, Connect to LLM, Run It.

We'll make a simple "Translator Agent" that turns English to Urdu. Follow along—it's beginner-friendly!

Prerequisites (Quick Check from Earlier Steps):

- UV installed and project set up.
- .env file with your API key (e.g., `GEMINI_API_KEY="your-key-here"` or OpenAI one).
- Python ready (UV handles packages).

Step 7: Install OpenAI Agents

- Open your VS Code terminal (Ctrl + `).
- Run: `uv add openai-agents`.
- This adds the library to your project fast (UV is quick!).
- What happens? It downloads the SDK and updates your project files (like `uv.lock`).
- Extra Tip from Me: If errors, check your internet or run `uv sync` first.

Step 8: Create Your First Agent

- In your main Python file (e.g., `main.py`), import the tools at the top:

```
from dotenv import load_dotenv # For loading .env
import os # For getting the key
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel # Import Agents stuff
```

- Now, create the agent:

```
agent = Agent(
    name="Translator", # Name: How the system knows this agent (like a label).
    instructions="You are a helpful assistant who always translates English sentences into clear and simple Urdu." # Job: What it does-be specific!
)
```

- Why Name and Instructions?
 - **Name:** Like a nickname—the framework calls it by this when needed.
 - **Instructions:** Tells the agent its role. Be clear!

- Analogy: Buying a phone. Option 1: "Give me a phone for 30,000 rupees." (Vague—might get anything.)
- Option 2: "Budget 30,000 rupees. Need good battery, camera, and storage." (Specific—gets what you want.)
- These are required—super important for any agent!
- Extra Tip: Agents can be anything—a math solver, joke teller. Change instructions to fit your idea.

Your Step 8 is done—the agent exists!

Step 9: Connect with Your LLM

- LLM means "Large Language Model"—a smart AI like Gemini or ChatGPT that understands human language and answers.
- Why connect? The agent needs a "brain" (LLM) to think.
- We're using free Gemini here (change to OpenAI if you want).
- Add this code after loading .env (before creating the agent):

```
load_dotenv() # Load secrets
gemini_api_key = os.getenv("GEMINI_API_KEY") # Get your key safely

if not gemini_api_key: # Check if key exists
    raise ValueError("No GEMINI_API_KEY! Add it to .env.")

external_client = AsyncOpenAI( # Connect to Gemini
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
)

model = OpenAIChatCompletionsModel( # Pick the model (Gemini Flash-fast/free)
    model="gemini-2.0-flash", # Or "gemini-2.5-flash" if available
    openai_client=external_client
)
```

- What it does: Verifies key, connects to Gemini's server, picks a model.
- Extra Tip: If using OpenAI, change base_url to OpenAI's and use their model (e.g., "gpt-4o-mini").

Step 9 done—agent has a brain!

Now we are at the **last step** → **Step 10: Run/Execute the Agent.**

- To run the agent, we need to use a function called **Runner**.
By default, **Runner is asynchronous**, but we can also make it **synchronous**. Since we are learning basics first, we will use the **synchronous** version.
- If we use `Runner.run()`, it will be asynchronous. To keep it synchronous, we use: `response = Runner.run_sync()`
- The `run_sync()` function takes 3 arguments:
 - Agent → the variable name of the agent you created (e.g. agent)
 - Prompt → what task you want the agent to do (e.g. translate a sentence)
 - Configuration → settings/config object
- Add this at the bottom:

```
response = Runner.run_sync(
    agent,
    input = 'My name is Sahir Ahmed Sheikh. I am a student of GIAIC',
    run_config = config
)
```

- **Synchronous vs Asynchronous?** (Quick Note—Full in Next PDF):
Synchronous means "wait for answer" (simple, like calling a friend and staying on line). Asynchronous is "do other things while waiting" (advanced, for big apps). We use `_sync` here for basics.
- Run in terminal: `uv run main.py`.
- Output (English to Urdu Example): Something like
 میرا نام ساحر احمد شیخ ہے۔ میں جی آئی اے آئی سی کا طالب علم ہوں۔
 → If Urdu looks weird in terminal (backwards): Copy to Notepad—it's right-to-left language issue. Terminal is left-to-right by default.
- Test Urdu to English: Change instructions to "Translate Urdu to English" and input Urdu. Output: "My name is Sahir Ahmed Sheikh. I am a student at GIAIC." (It even fixes grammar—like "of" to "at" for better English!)

Full Code Together (Copy This to main.py):

```
from dotenv import load_dotenv
import os
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel, RunConfig

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

# Check if the API key is present; if not, raise an error
if not gemini_api_key:
    raise ValueError("GEMINI_API_KEY is not set. Please ensure it is defined in your .env file.")

# Reference: https://ai.google.dev/gemini-api/docs/openai
external_client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

model = OpenAIChatCompletionsModel(
    model="gemini-2.0-flash",
    openai_client=external_client
)

config = RunConfig(
    model=model,
    model_provider=external_client,
    tracing_disabled=True
)

agent = Agent(
    name = "Translator",
    instructions = "You are a helpful assistant who always translates Urdu sentences into clear and simple English."
)

response = Runner.Run_sync(
    agent,
    input = 'میرا نام سہار احمد شیخ ہے۔ میں جی اے اے ای سی کا طالب علم ہوں۔',
    run_config = config
)

print(response)
```

How It Works (Full Breakdown):

- Loads key from .env.
- Checks if key exists (safety!).
- Connects to Gemini (verifies key, picks model).
- Creates agent with name/instructions.
- Runs: Sends prompt, gets response from LLM.
- Prints output.
- Why Agentic AI? Simple AI just repeats what you say. Agentic AI enhances (e.g., fixes grammar without asking—like "of GIAIC" to "at GIAIC").

- Extra Tip: Test flips—English to Urdu first (output might look reversed in terminal; paste to Notepad). For apps like Streamlit, it shows perfectly.

Your first agent is ready! Run it and tweak instructions (e.g., make a joke teller). If errors: Check key, imports, or run `uv sync`. Practice makes it fun.

Step 05: Model Configuration

In this step, we're going to learn how to set up (configure) the "brain" of your AI agents—the Large Language Model (LLM) like Gemini or GPT. The OpenAI Agents SDK starts with OpenAI as the default, but you can switch to others (like Gemini) easily. Why? Different models are good for different jobs—some are faster, cheaper, or better at certain tasks.

I'll explain everything in super simple words, with step-by-step code. We'll cover three levels: Agent Level (best for most cases), Run Level, and Global Level. I've added extras like why to choose each level and common mistakes.

Practice by copying code into VS Code!

Why Configure Models?

- **Default:** SDK uses OpenAI (easy start).
- **Custom:** For Gemini or others, tell it your API key and URL.
- **Levels:** Set at Agent (per agent), Run (per run), or Global (for everything).
- **Analogy:** Like choosing a teacher—pick one for math (Agent Level), one session (Run Level), or the whole school (Global Level).
- **Extra Tip:** Always use Agent Level first—it's flexible. Global is for big projects where everything uses one model.

1. Agent Level Configuration

This is the best way: Each agent gets its own model. Great if one agent needs a fast model (like Gemini Flash) and another needs a smart one (like GPT-4).

- **Why?** Custom fit—agents work independently.
- **How?** Set "model" when creating the agent.

- It's async by default (good for advanced stuff), but simple to use.

Steps:

1. Import tools (add to top of main.py).
2. Load your API key (from .env).
3. Create the client (connection to Gemini).
4. Make the agent with its model.
5. Run it async (use **await** for waiting).

Full Code Example (Async—Copy This):

```
import asyncio # For async running
from openai import AsyncOpenAI # Connection tool
from agents import Agent, OpenAIChatCompletionsModel, Runner, set_tracing_disabled # Agents stuff
from dotenv import load_dotenv # For secrets
import os

load_dotenv() # Load .env
gemini_api_key = os.getenv("GEMINI_API_KEY") # Get key

if not gemini_api_key: # Safety check
    raise ValueError("No GEMINI_API_KEY! Add to .env.")

set_tracing_disabled(disabled=True) # No extra logs

# Connect to Gemini
client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
)

async def main(): # Async function
    # Agent with its own model
    agent = Agent(
        name="Poet", # Name
        instructions="You are a fun poet. Write short poems.", # Job
        model=OpenAIChatCompletionsModel( # Custom model here!
            model="gemini-2.0-flash", # Fast Gemini model
            openai_client=client
        )
    )

    # Run it
    result = await Runner.run(
        agent,
        "Tell me about recursion in programming." # Question
    )
    print(result.final_output) # Show answer

if __name__ == "__main__":
    asyncio.run(main()) # Start async
```

Output (Example Poem):

```
Code calls itself deep,  
Loops in endless mystery,  
Base case breaks the spell.
```

What Happens?

- Connects to Gemini.
- Agent uses its own model.
- Runs async (waits smartly).
- Extra Tip from Me: Async is like cooking while waiting for water to boil—efficient. If errors: Check key or model name (e.g., "gemini-2.5-flash" if available).

Common Mistake: Wrong base_url—copy exactly for Gemini.

2. Run Level Configuration

Set the model for one "run" (session). Good if testing different models without changing agents.

- Why? Quick changes per test.
- How? Use "run_config" when running.
- Sync by default (simple, like Step 04).

Steps:

1. Import and load key.
2. Create client and model.
3. Make a basic agent (no model yet).
4. Add config when running.

Full Code Example (Sync—Copy This):

```
from dotenv import load_dotenv
import os
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel
from agents.run import RunConfig # For run settings

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

if not gemini_api_key:
    raise ValueError("No GEMINI_API_KEY! Add to .env.")

# Connect to Gemini
external_client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
)

# Model setup
model = OpenAIChatCompletionsModel(
    model="gemini-2.0-flash", # Your model
    openai_client=external_client
)

# Run settings
config = RunConfig(
    model=model, # Model here
    model_provider=external_client, # Connection
    tracing_disabled=True # No logs
)

# Basic agent (no model yet)
agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant."
)

# Run with config
result = Runner.run_sync(
    agent,
    "Hello, how are you?", # Question
    run_config=config # Settings applied here
)

print(result.final_output) # Answer
```

Output (Example):

I'm doing well, thank you! How can I assist you today?

What Happens?

- Model set only for this run.
- Sync: Waits step-by-step.
- Extra Tip: Use for experiments—swap models easily. Mistake: Forget config—falls back to default (OpenAI).

3. Global Level Configuration

Set one model for EVERYTHING in your code. Good for simple projects using the same model everywhere.

- Why? Easy setup—no repeats.
- How? Use "set_default" functions at the start.

Steps:

1. Import extras like set_default_openai_client.
2. Set global settings early.
3. Agents use it automatically.

Full Code Example (Global—Copy This):

```
from dotenv import load_dotenv
import os
from agents import Agent, Runner, AsyncOpenAI, set_default_openai_client, set_tracing_disabled, set_default_openai_api

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

if not gemini_api_key:
    raise ValueError("No GEMINI_API_KEY! Add to .env.")

set_tracing_disabled(True) # No logs
set_default_openai_api("chat_completions") # API type

# Global connection
external_client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
)
set_default_openai_client(external_client) # Set for all

# Agent uses global model
agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant",
    model="gemini-2.0-flash" # Global applies
)

# Run normally
result = Runner.run_sync(agent, "Hello")

print(result.final_output)
```

Output (Example):

```
Hello! How can I help you?
```

What Happens?

- One setup for all agents/runs.
- Extra Tip: Best for small apps. Mistake: Forgets to set early—code uses default OpenAI.

Which Level to Choose?

- **Agent Level:** Best—custom per agent (your preference).
- **Run Level:** For testing one session.
- **Global Level:** For simple, uniform projects.
- Analogy: Agent = Pick shoes per person. Run = Per walk. Global = One size for all.
- Extra from Me: Mix them! Agent overrides Run/Global. Start with Agent for flexibility.

Common Mistakes: Wrong API key (test with print), missing imports (check errors), or forgetting .env load.

You've configured models! Practice by changing models (e.g., "gemini-2.5-flash").

Note: I have made every effort to explain this to you in simple wording.

If you find it difficult to understand in English, you can provide the entire PDF, a specific line, or even a single word to ChatGPT and get it translated into Urdu or any other language to understand it more clearly.

Insha'Allah, in the next PDF, I will cover the complete Agentic AI syllabus step-by-step in simple terms. For example, Step 06 – Basic Tools, Step 07 – Model Settings, and so on, until the entire syllabus is explained, preparing you for both Level 1 and Level 2 of the Agentic AI test.